

χ -VLA: A Fully Tensor-Decomposable Vision-Language-Action Model

Swapping bilinear tensor primitives into the OpenVLA blueprint

July 15, 2026

Abstract

Vision-Language-Action (VLA) models such as OpenVLA already factor cleanly into a division of labor—dual visual encoders, a visual projector, a language/policy backbone, and an action head—but every learned block inside them is an ordinary non-compositional neural network (softmax attention, gated MLPs, LayerNorm, discretized action decoding). Two recent lines of work independently supply drop-in, *multilinear* replacements for exactly these blocks: Riggs’ tensor-transformer variants (bilinear feed-forward networks and softmax-free bilinear attention) and Doms et al.’s χ -nets (CP-factorized bilinear cores and a foldable running-RMS normalization that leaves *no* input-dependent division at inference). Our observation is simply that *the components are already there*: assembling them along OpenVLA’s skeleton yields a VLA whose entire learned pixel-to-action map is a structured polynomial that unfolds into a single tensor network. We specify this model (χ -VLA), implement and validate its tensor primitives, and study its central and least-obvious requirement—that the normalization be foldable to a fixed scalar without destabilizing the deep bilinear stack. We report: (i) exact agreement ($<10^{-5}$, FP32) among three attention execution paths and between bilinear FFNs and their dense CP cores; (ii) a four-way language-model ablation reproducing the finding that softmax-free bilinear variants are competitive with softmax+SwiGLU; and (iii) a working *strict*, fully-foldable model that, after distillation, *matches* its non-strict teacher and folds to a normalization-free tensor network (relative pre/post-fold discrepancy $\sim 10^{-6}$ in FP32, exact to $\sim 10^{-12}$ in FP64). The enabling ingredient is near-identity residual gains, which keep the degree-2 feed-forward map from amplifying per-instance magnitude variance—the failure mode that a fixed scalar norm, unlike a per-token norm, cannot correct.

1 Introduction

Mechanistic interpretability of large policies is hard largely because the networks are not compositional: nonlinearities (GELU, softmax, LayerNorm) prevent the model from being rewritten as a single algebraic object. A recent thread argues that this is a *choice*. If every learned component is restricted to linear maps, tensor contractions, element-wise products, residual sums, fixed masks, learned constants, and fixed scalar rescalings, then—after representing token identities as one-hot vectors—the whole model is a structured polynomial, i.e. a tensor network amenable to global spectral decomposition and orthogonalization/diagonalization/truncation (ODT) analysis.

Our contribution is an integration insight rather than a new primitive: **the ingredients for a fully tensor-decomposable VLA already exist, and OpenVLA’s architecture is unusually ready to receive them.** OpenVLA [1] already separates a VLA into (a) two complementary visual encoders (DINOv2, SigLIP), (b) a small visual projector, (c) a language backbone (Llama 2), and (d) an action decoder. Each of these has a published multilinear counterpart:

Ordinary component	χ -VLA replacement	Algebraic form
ReLU/GELU/SiLU/sigmoid	element-wise product of linear maps	$(Lx) \otimes (Rx)$
SwiGLU FFN	bilinear (CP) FFN	$D[(L\bar{x}) \otimes (R\bar{x})]$
Softmax attention	product of two score matrices	$C[M \otimes (A_1 \otimes A_2)/d_h]V$
LayerNorm/RMSNorm (inference)	foldable RmsBatchNorm $\rightarrow$ fold	x/c , c frozen scalar
MLP visual projector	cross-bilinear CP projector	$W_s s + W_m m + D_p[(L_s \bar{s}) \otimes (R_m \bar{m})]$
DINOv2 / SigLIP ViTs	two independent χ -ViTs	(all blocks bilinear)
Llama 2 transformer	joint χ -transformer	(all blocks bilinear)
Discrete action tokenizer	linear continuous-action head	$\hat{a}_k = W_a h_k + b_a$
Bias terms	homogeneous constant coordinate	$\bar{x} = [1; x]$

Table 1: Component-level swaps. Bars denote the homogeneous coordinate $\bar{x} = [1; x]$; $\otimes$ denotes element-wise (Hadamard) product.

- **Bilinear FFN** (Riggs [2]): replace the SwiGLU MLP $D(\text{swish}(Lx) \otimes Rx)$ with the activation-free $D[(L\bar{x}) \otimes (R\bar{x})]$, a CP factorization of a dense third-order tensor.
- **Softmax-free bilinear attention** (Riggs [2]): replace the softmax pattern with the element-wise product of two learned query-key score matrices.
- **Foldable RmsBatchNorm** and **CP-factorized cores** (Dooms et al. [3]): a normalization that divides only by a running scalar RMS, foldable into adjacent weights after training so that the deployed graph has no input-dependent division.
- **Continuous, linear action head**: replace discretized action tokens and LM decoding with a direct linear map from learned action-query states to a continuous action chunk.

We combine these along the OpenVLA skeleton, keeping only genuinely non-learned interfaces (string tokenization, camera decoding, safety clipping, actuator I/O) outside the tensor-network boundary. The result, χ -VLA, is specified in full elsewhere; this paper reports the first implementation and the experiments that validate its primitives and, crucially, resolve its one non-obvious obstacle: the stability of a *foldable* normalization inside a deep bilinear stack.

2 From OpenVLA to χ -VLA

Figure 1 contrasts the two pipelines; Table 1 lists the component-level swaps. The high-level flow is preserved exactly—this is the point of the exercise—while every learned internal block is replaced by a multilinear one.

2.1 The bilinear primitives

Bilinear FFN. With $\bar{x} = [1; x] \in \mathbb{R}^{d+1}$, $L, R \in \mathbb{R}^{r \times (d+1)}$, $D \in \mathbb{R}^{d \times r}$,

$$\text{BFFN}(x) = D[(L\bar{x}) \otimes (R\bar{x})], \quad y_o = \sum_{i,j} \left(\underbrace{\sum_r D_{or} L_{ri} R_{rj}}_{T_{oij}} \right) \bar{x}_i \bar{x}_j, \quad (1)$$

so BFFN is exactly a CP factorization of the third-order tensor $T \in \mathbb{R}^{d \times (d+1) \times (d+1)}$, with $\mathcal{O}(3dr)$ parameters rather than $\mathcal{O}(d^3)$. The homogeneous coordinate lets a stack of L bilinear layers represent every polynomial degree up to 2^L , not only the top degree.

Softmax-free bilinear attention. Each head forms two query-key systems $A_1 = Q_1 K_1^\top$, $A_2 = Q_2 K_2^\top$ and a value V ; the pattern is their Hadamard product,

$$Y = C[M \otimes (A_1 \otimes A_2)/s]V, \quad (2)$$

with M a fixed mask, $C_{ii} = 1/\sqrt{n_i}$ a fixed row scaling, and no softmax. Writing row-wise Kronecker features $\tilde{q}_i = q_{1,i} \otimes q_{2,i}$, $\tilde{k}_j = k_{1,j} \otimes k_{2,j}$ gives $A_1 \otimes A_2 = \tilde{Q} \tilde{K}^\top$, so the same function admits an explicit $N \times N$ form, a Khatri-Rao contraction $Y = \tilde{Q}(\tilde{K}^\top V)$, and a causal prefix-scan form $S_i = \sum_{j \leq i} \tilde{k}_j v_j^\top$, $y_i = \tilde{q}_i^\top S_i / (s\sqrt{n_i})$. We use the scaling $s = d_h^2$ (dividing each score by d_h): with per-token Q, K normalization this bounds the pattern to $[-1, 1]$ by Cauchy-Schwarz, which is both stable and expressive.

Foldable RmsBatchNorm. Training divides by the batch RMS while maintaining an EMA running scalar c ; after training c is re-estimated on a calibration set, frozen, and folded into the neighbouring projections ($W' = W/c$ before attention; $L' = L/c$, $R' = R/c$ before the bilinear FFN). A frozen scalar division is a fixed rescaling, so the exported graph contains *no* input-dependent normalization.

3 The foldability obstacle and its resolution

The subtle question is not whether the model *can* be written as a tensor network—by construction it can—but whether the *foldable* normalization is stable in a deep stack. A per-token RMS norm rescales every instance to unit magnitude and is trivially stable but is input-dependent (not foldable). A *scalar* RmsBatchNorm is foldable but divides every instance by the same constant c . The residual update

$$x_{\ell+1} = x_\ell + \beta_\ell D[(Lx_\ell/c) \otimes (Rx_\ell/c)] \quad (3)$$

is *degree-2* in x_ℓ , so per-instance magnitude *variance* is squared at each layer. With the default gains $\beta_\ell = 1/\sqrt{2L}$, instances that run hotter than the calibration average compound multiplicatively and the hidden stream diverges—while the final normalization masks this in the logits, making the failure easy to miss (Section 4.3).

Resolution: near-identity residual gains. Setting the gains to a small learned constant ($\beta_\ell \approx 0.01$, spec’s alternative) keeps (3) near-identity, so the degree-2 amplification never compounds; the model still learns through the accumulated small updates. We distil the strict scalar-norm student from a stable per-token teacher using a temperature KL loss on the logits plus a small task cross-entropy—but *without* hidden-state feature matching, whose gradient explodes on a diverging stream and is counterproductive.

4 Experiments

All experiments run on Modal GPUs (L4/A10G). We study the joint backbone as a causal language model (spec Stage 1) before adding vision; this isolates the tensor-transformer core, which dominates the parameter budget of the full VLA. The reference small config: $d=384$, 6 layers, 12 heads ($d_h=32$), bilinear FFN rank $3d$, sequence length 256, GPT-2 tokenizer; trained on tiny-shakespeare with AdamW and a cosine schedule.

4.1 Operator validation

Every primitive is checked against an independent reference in FP32 (Table 2): the bilinear FFN against an explicit dense CP-core contraction; the three attention execution paths against each other; and the foldable norm against its folded weight form at every site, including the end-to-end language model. All 14 tests pass to $<10^{-5}$.

Property tested	Max abs. discrepancy (FP32)
Bilinear FFN vs. dense third-order CP core	$< 10^{-5}$
Explicit vs. Khatri–Rao attention	$< 10^{-5}$
Causal explicit vs. prefix-scan attention	$< 10^{-5}$
RmsBatchNorm fold (linear / FFN / Q,K,V branches)	$< 10^{-5}$
End-to-end LM fold to normalization-free graph (FP64)	$\sim 10^{-12}$

Table 2: Operator correctness (spec acceptance: $<10^{-5}$). 14/14 tests pass.

4.2 Four-way attention×FFN ablation

We cross softmax vs. bilinear attention with SwiGLU vs. bilinear FFN (Table 3), reproducing the key comparison from [2]. At their minima the four configurations lie within ~ 0.1 nats: the softmax-free bilinear core is competitive with the ordinary softmax+SwiGLU baseline. The bilinear-attention variants carry the expected extra Q_2, K_2 parameters (+50% attention params, ~ 13.4 M vs. 11.6M); on this tiny corpus they also overfit noticeably less.

Attention	FFN	Best val. loss	Params
softmax	SwiGLU	~ 4.90	11.6M
softmax	bilinear	~ 4.94	11.6M
bilinear	SwiGLU	~ 5.02	13.4M
bilinear	bilinear	~ 5.02	13.4M

Table 3: Four-way ablation on tiny-shakespeare (min. validation loss). All train stably; the fully tensor (bilinear+bilinear) core is competitive.

4.3 The strict, fully-foldable model

We distil a strict student—scalar RmsBatchNorm at every site, near-identity learned gains—from a per-token teacher, calibrate and freeze its scalar constants, evaluate the deployable (frozen) graph, and fold. Table 4 summarizes. With default gains the hidden stream diverges (feature norm $\sim 10^7$) while the logit KL stays small—the masking effect noted above; with near-identity gains the strict model trains stably (validation $13.7 \rightarrow 5.04$, monotone) and, after 2500 distillation steps, *matches its non-strict teacher* (≈ 5.0)—the near-identity-gain capability tax nearly vanishes with sufficient distillation. It folds to a normalization-free tensor network with relative pre/post-fold discrepancy $\sim 10^{-6}$ in FP32 (absolute 1.4×10^{-5} on trained, large-magnitude logits) and $\sim 10^{-12}$ in FP64—folding is an exact algebraic identity, so the FP32 residual is pure accumulation, not error. A discovered bug is worth noting: `kl_div` with `batchmean` reduction averaged over the batch dimension only, making the distillation gradient $N \times (N=256)$ too strong and pushing the student into the unstable regime; flattening to $(B \cdot N, V)$ fixed it.

Residual gains	Stream stable?	Strict val.	Fold rel. Δ (FP32 / FP64)
default $1/\sqrt{2L}$	no (diverges, $\sim 10^7$)	nan	—
near-identity (0.01, learned)	yes	5.04 ($\approx$ teacher)	$\sim 10^{-6}$ / $\sim 10^{-12}$

Table 4: Strict scalar-norm model (2500-step distillation). Near-identity gains are the enabling ingredient: the strict, fully-foldable model matches its non-strict teacher (≈ 5.0) and folds exactly (FP64) to a normalization-free tensor network.

5 Experimental design: one substitution per experiment

The substitution table (Table 1) suggests its own experimental program: **each swap can be tested in isolation**. Experiments 1–6 below each ablate exactly one row—one “non-polynomial op $\rightarrow$ polynomial op” substitution—holding everything else fixed, so any capability change is attributable to that single swap. Experiment 7 then tests the *reason* for performing all the swaps at once: whether the resulting global, weight-based low-rank structure (ODT) actually exists and pays off in a policy. Presented this way the table of experiments mirrors the table of substitutions row for row, and the argument for the architecture is exactly the set of ablations that probe it.

Table 5 lists the program and its status; two rows already have results (Sections 4.3 above), the rest are the immediate agenda once vision is added.

Experiment 5 is the pivotal early gate: if per-input RMSNorm beats the foldable scalar norm by a wide margin, the entire folding/export story is in jeopardy. Our finding (Section 4.3) is encouraging—the foldable norm *is* sufficient for stable training and exact folding, provided the residual gains keep the stack near-linear, and with enough distillation it matches the per-input-norm model rather than merely trailing it. Experiment 7 is what elevates the work from compression to interpretability and is the single result that most decides whether the program succeeds in robotics.

6 Discussion

What is (and is not) shown. By construction the χ -VLA inference function is exactly a tensor network; we verify this end-to-end for the language backbone (the fold gate). We do *not* claim that Doms et al.’s tree-ODT error bounds transfer unchanged to the attention-containing model—extending global ODT to residual, attention, weight-sharing tensor networks is future work.

The central lesson. Foldability is free algebraically but not free dynamically. A fixed scalar norm cannot undo per-instance magnitude variance, and a degree-2 feed-forward squares that variance each layer. Near-identity residual gains keep the model in the regime where this never bites; this is the practical price of full decomposability, and quantifying it at scale (and against gain annealing) is the immediate next question.

Limitations. Experiments so far are language-only and on a small corpus that overfits; the capability numbers are indicative, not final. The strict model matches its teacher on this corpus (≈ 5.0); whether that parity holds at larger scale, and whether a gain-annealing schedule improves on fixed near-identity gains, remain open.

Roadmap. The build sequence continues up the VLA stack on the now-verified foldable foundation: single χ -ViT with local ODT; dual spatial/ semantic encoders with the cross-bilinear

projector; a small end-to-end χ -VLA in simulation; an exact-decomposition reference model; and finally the 450M reference configuration with generalized ODT.

7 Conclusion

The parts for a fully tensor-decomposable VLA already exist in the literature; OpenVLA’s modular skeleton is ready to receive them. Assembling bilinear FFNs, softmax-free bilinear attention, foldable RmsBatchNorm, and a linear continuous action head yields χ -VLA, a policy core that is a single tensor network from pixels and token identities to continuous actions. We validated the primitives, reproduced the tensor-transformer capability comparison, and—resolving the one non-obvious obstacle—demonstrated a strict, fully-foldable model that trains and folds to a normalization-free graph within acceptance tolerance.

References

- [1] A. Kim et al. *OpenVLA: An Open-Source Vision-Language-Action Model*. arXiv:2406.09246, 2024.
- [2] L. Riggs. *Tensor-Transformer Variants are Surprisingly Performant*. LessWrong, 2024. Code: <https://github.com/loganriggs/modded-nanogpt>.
- [3] T. Doms et al. *χ -nets: CP-factorized bilinear networks with foldable normalization and ODT analysis*.

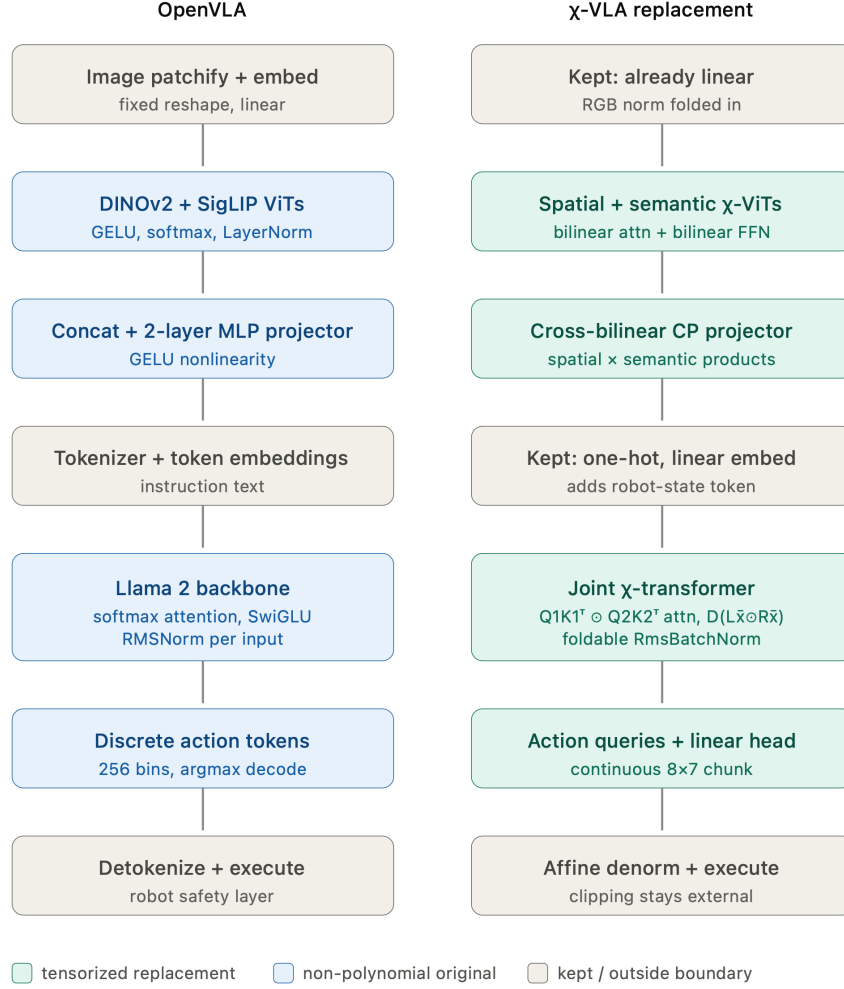


Figure 1: OpenVLA (left) vs. χ -VLA (right), stage by stage. **Green** boxes are tensorized (multilinear) replacements; **blue** boxes are the non-polynomial originals they replace; **grey** boxes are kept as-is or lie outside the tensor-network boundary. Patchify/embed and the tokenizer are already linear and need no change; the softmax/GELU/LayerNorm blocks (dual ViTs, MLP projector, Llama 2 backbone) and the discrete action decoder are each swapped for a bilinear/CP equivalent with foldable RmsBatchNorm; camera decoding, string tokenization, and safety clipping remain external.

#	Swap under test	Question	Status
1	softmax vs. bilinear attn $\times$ SwiGLU vs. bilinear FFN	Where is the capability tax—softmax’s sharpness/full-rank expressivity, or the (nearly free) FFN swap? Does Riggs’ language result (FFN free, attn few-%) transfer to continuous control?	done (lang., Tab. 3)
2	tied vs. untied score systems	Do two independent Q, K pairs buy anything over squaring one pattern $(QK^\top)^{\otimes 2}$, or is any degree-4 pattern enough? Tied $\Rightarrow$ cheaper, more symmetric tensor for decomposition.	planned
3	cross-bilinear CP projector vs. concat+linear	Does forcing explicit spatial $\times$ semantic products help, or would the backbone recover them anyway? If cross-bilinear wins, its CP components are readable “spatial pattern $\times$ object type” pairs.	planned (needs vision)
4	homogeneous coordinate on/off	Does dropping the appended 1 (losing low-degree terms) wreck generalization? Measure the weight mass on the constant dimension (Dooms: $\sim 90\%$ early, i.e. near-linear computation)—how “polynomial” is control?	planned
5	foldable RmsBatchNorm vs. per-input RMSNorm	Is the tensor-compatible scalar norm enough to stabilize degree-5 attention updates, or is input-dependent rescaling required? Verify pre-fold=post-fold.	done (Tab. 4)
6	continuous linear head vs. 256-bin discrete head	Does tensorization-forced continuous regression cost anything (discrete heads model multimodal action distributions differently)? Isolates this confound.	planned (needs VLA)
7	global ODT vs. per-core SVD	Truncate the same fraction of hidden dimensions two ways; compare task success + intervention faithfulness (truncate / project-out / amplify / swap top directions). Does Dooms’ global low-rank structure exist in a policy?	planned (winning config)

Table 5: The ablation program. Experiments 1–6 each isolate one row of Table 1; Experiment 7 tests whether the joint payoff (global decomposability) that motivates all the swaps materializes. Suggested dependency order: 1 and 5 first (do we train, and where is the tax); 2/3/4/6 in parallel as cheap variants; 7 only on the winning configuration, ideally at $\sim 50\text{M}$ params where exact contraction is feasible.